

A



UD.1: INTRODUCCIÓN A GIT/GITHUB

Implantación de Aplicaciones Web (IAW)

CFGS ASIR

Mara Vañó Alonso

Curso 2024-2025

Índice

Índice	2
GIT	3
Introducción	3
INSTALACIÓN DE GIT	3
USO BÁSICO DE GIT	3
AÑADIR FICHEROS	4
MODIFICAR UN FICHERO	4
ESTADO DEL REPOSITORIO	4
OTRAS OPERACIONES.....	4
VOLVIENDO A VERSIONES ANTERIORES	5
GITHUB	5
Introducción	5
CLONAR REPOSITORIO	5
https://docs.github.com/es/repositories/creating-and-managing-repositories/cloning-a-repository	5
SINCRONIZANDO REPOSITORIO LOCAL Y REMOTO	5
AÑADIENDO UN REPOSITORIO REMOTO	6
RAMAS	6
RAMIFICAR	6
SINCRONIZAR RAMAS CON EL REPOSITORIO REMOTO	7
ELIMINAR UNA RAMA	7
FUSIÓN DE RAMAS	7
PULL REQUEST	8
PASOS:.....	8
SINCRONIZACIÓN DE NUESTRO FORK CON EL REPOSITORIO ORIGINAL	8



Introducción

Git: es un software de control de versiones diseñado por Linus Torvalds. Nos ayuda a llevar registro de los cambios de los ficheros de un repositorio (directorio).

INSTALACIÓN DE GIT

Instalación de git

```
$ apt install git
```

Configuración de git.

```
$ git config --global user.name "Mara Vaño"
```

```
$ git config --global user.email m.vanoalonso@correo.com
```

USO BÁSICO DE GIT

Creamos un directorio

```
$ mkdir curso-de-git
```

```
$ cd curso-de-git
```

Inicializamos el repositorio

```
$ git init
```

Inicializado repositorio Git vacío en ...

Se ha creado el directorio .git. El fichero de configuración del repositorio en. git/config

AÑADIR FICHEROS

Añadimos un nuevo fichero al repositorio:

```
$ echo "Esto es una prueba">ejemplo.txt  
$ git add ejemplo.txt
```

Confirmamos los cambios:

```
$ git commit -m "He creado el fichero ejemplo.txt"
```

MODIFICAR UN FICHERO

Al modificar un fichero, podemos confirmar el cambio, pero tenemos que usar la opción **a:**

```
$ nano ejemplo.txt  
$ git commit -am "He modificado el fichero ejemplo.txt"
```

ESTADO DEL REPOSITORIO

Estado del repositorio

```
$ git status
```

Listado de commits:

```
$ git log
```

OTRAS OPERACIONES

Cambiar nombre a fichero:

```
$ git mv ejemplo.txt ejemplo2.txt  
$ git commit -am "He cambiado el nombre del fichero"
```

Eliminar un fichero

```
$ git rm ejemplo2.txt
```

```
$ git commit -am "He borrado el fichero ejemplo2"
```

VOLVIENDO A VERSIONES ANTERIORES

Podemos volver al estado de un repositorio en un commit anterior.

Si queremos volver a un commit anterior sin perder las modificaciones posteriores:

```
$ git checkout <id del commit>
```

Para volver a la última versión:

```
$ git switch -
```

Si quiero eliminar los últimos 3 commits:

```
$ git reset --hard HEAD~3
```

The logo consists of the word "GITHUB" in a bold, orange, sans-serif font. The letters are contained within a thin orange rectangular border.

Introducción

GitHub: Es un servicio que nos permite guardar repositorios Git en un servidor remoto.

CLONAR REPOSITORIO

<https://docs.github.com/es/repositories/creating-and-managing-repositories/cloning-a-repository>

SINCRONIZANDO REPOSITORIO LOCAL Y REMOTO

Para subir los cambios desde nuestro repositorio local al remoto usamos:

```
$ git push
```

Para descargar los cambios desde nuestro repositorio remoto al local usamos:

```
$ git pull
```

AÑADIENDO UN REPOSITORIO REMOTO

Vamos a crear un repositorio local, pero el nombre de la rama tiene que ser **main:**

```
$ git init -b main
```

Si ya tenemos un repositorio, podemos cambiar el nombre de la rama:

```
$ git branch -M main
```

Creamos un repositorio vacío (Sin _chero README.md**) en GitHub.**

Añadimos el repositorio remoto y sincronizamos:

```
$ git remote add origin git@github.com:mvano/prueba.git
```

```
$ git push -u origin main
```

RAMAS

Una **rama** representa una línea independiente de desarrollo, es como crear una nueva área de trabajo que tendrá su historial propio de commits.

Para listar las ramas locales ejecuta:

```
$ git branch
```

```
$ * main
```

La rama en la que estás trabajando actualmente se señala con un asterisco *.

GitHub llama a la rama principal **main**. En repositorios antiguos se llama **master**.

En la rama principal es donde suele encontrarse la **última versión** del repositorio.

RAMIFICAR

Creamos una nueva rama, para realizar cambios al repositorio (sin tocar el estado de la rama

principal).

Si los cambios son correctos, tenemos que **fusionar** la rama creada con la rama principal.

Para crear una nueva rama:

```
$ git branch [rama]
```

Para pasar a la nueva rama utiliza el comando:

```
$ git checkout [rama]
```

La nueva rama tiene los mismos ficheros que en la rama principal.

Todos los cambios que realices en los ficheros en esta rama no se verán en la rama principal.

Truco: con el comando **git checkout -b [rama]** se crea una nueva rama y te posicionas en ella.

Con el comando **git branch -v** se ve el último commit de cada rama. Comprueba que coinciden el último commit en las dos ramas.

Truco: Puedes usar una extensión de tu shell (bash, zsh, . . .) para que te muestre en el prompt la rama en la que estás trabajando.

SINCRONIZAR RAMAS CON EL REPOSITORIO REMOTO

Las ramas no se crean automáticamente en GitHub, hay que realizar un push para crearlas en remoto.

Nota: origin es el nombre del repositorio remoto.

```
$ git push origin [rama]
```

ELIMINAR UNA RAMA

Es bastante frecuente crear una rama, hacer los cambios que sean necesarios, unirla a una rama principal y después eliminar la rama que se había creado.

```
$ git branch -d [rama]
```

FUSIÓN DE RAMAS

Una vez que has trabajado en una rama, lo normal es querer incorporar los cambios a la rama

principal. Para **unir/fusionar** una rama a la principal:

```
$ git checkout main
```

```
$ git merge [rama]
```

PULL REQUEST

Un **Pull Request** es una petición que se hace al propietario de un repositorio original para que este último incorpore los cambios que se sugieren.

Los **Pull Requests** se han convertido en la forma más habitual de colaborar en proyectos de software libre.

PASOS:

1. El usuario tiene que hacer un **fork** del repositorio al que quiere contribuir.
2. **Clonar** el repositorio.
3. **Crear una nueva rama**, donde realizaremos los cambios que posteriormente propondremos como cambios.
4. Subimos los cambios a nuestro repositorio, creando una rama en el repositorio remoto.
5. Crear el Pull Requests desde la página de GitHub.
6. El dueño del repositorio al que hemos realizado el PR tendrá que validar el cambio.
7. Y decide si lo **acepta**, haciendo una unión con la rama principal, o lo **rechaza**.

SINCRONIZACIÓN DE NUESTRO FORK CON EL REPOSITORIO ORIGINAL

Al hacer el fork del repositorio original hacemos una copia completa en un determinado momento, pero ese repositorio puede ir cambiando.

Esos cambios no se verán reflejados en nuestro **fork** de forma automática.

Antes de proponer un cambio (hacer un Pull Request), tener nuestro fork lo más actualizado posible para que no tengamos problemas que produzcan conflictos.

1. Nos posicionamos en la rama principal del fork.
2. Añadimos un repositorio remoto apuntando al repositorio original (**upstream**).

3. Buscamos los posibles cambios que ha tenido el repositorio original:

```
$ git fetch upstream
```

4. fusionamos los cambios del repositorio original que están en la rama principal.

```
$ git merge upstream/main
```